CRAIC-百度智能云智能服务机器人赛

机械臂调节教程

目录

一、 开源资料说明 3
1.1. 下载地址 3
1.2. 资料说明 3
二、 机械臂调试助手使用 3
2.1. 软件安装 3
2.2. 固件升级 4
三、 动作示教 5
四、 动作库备份 7
五、 关节标定 8
六、 软件控制 11
6.1. 动作库执行 11
6.2. 单关节控制 12
6.3. 机械爪运动控制 13

一、 开源资料说明

1.1. 下载地址

CRAIC-百度智能云智能服务机器人赛-机械臂调节教程

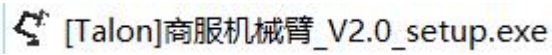
1.2. 资料说明

开源资料分为 3 个文件，分别为机械臂调试助手安装包、机械臂升级固件、机械臂示教说明视频。

二、 机械臂调试助手使用

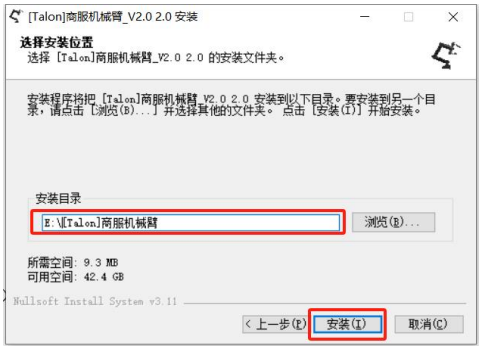
2.1. 软件安装

（1） 双击机械臂调试助手安装包



（2） 下一步

（3） 可自定义安装路径，安装



（4） 完成

2.2. 固件升级

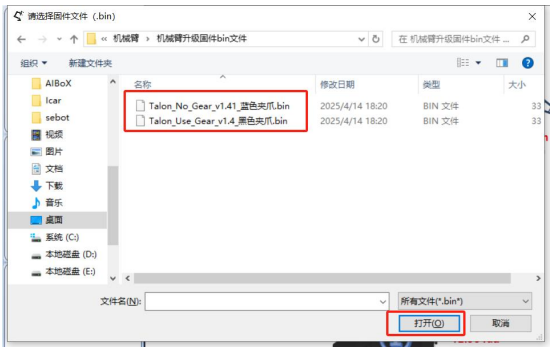
固件升级是为了适配 25 年新赛题的控制动作，支持机械臂每个关节的单独控制，以及动作库的导入导出，请先进行机械臂固件升级。

（1） 将机械臂 USB 数据线连接到电脑

（2） 软件自动获取机械臂串口信息并连接

（3） 点击选择固件文件

（4） 根据机械臂夹爪颜色，选择对应固件文件，点击打开



（5） 点击软件升级

（6） 等待升级完成

（7） 重启机器人

三、 动作示教

机械臂保存了机械臂复位、机械臂伸展、机械臂收缩、机械臂放置、自定义轨迹五个动作，机器人运行过程中通过动作库中的五个动作配合单关节控制完成所有任务，选手可通过动作示教功能自定义动作库中的五个动作（可参考演示视频）。

（1） 将机械臂 USB 数据线连接到电脑

（2） 软件自动获取机械臂串口信息并连接

（3） 选择动作库中的动作

（4） 点击启动示教

（5） 机械臂蜂鸣器发出鸣响后，用手拉动机械臂做出对应动作

（6） 点击停止示教

（7） 机械臂蜂鸣器发出鸣响后，动作成功保存

（8） 进行收缩、放置两个的标定是，可先使用执行动作功能，将机械臂调整至伸展、收缩，保证当前动作的起点是上一个动作的终点

（9） 点击恢复默认可将动作库恢复到初始状态

四、 动作库备份

动作库示教会覆盖当前动作，使用动作库备份功能可将机械臂当前动作导出保存，方便选手持续优化动作。

（1） 将机械臂 USB 数据线连接到电脑

（2） 软件自动获取机械臂串口信息并连接

（3） 选择动作库中的动作

（4） 点击读取动作库

（5） 选择文件路径，可自定义名称

（6） 使用写入动作库功能可将保存的动作写入机械臂

五、 关节标定

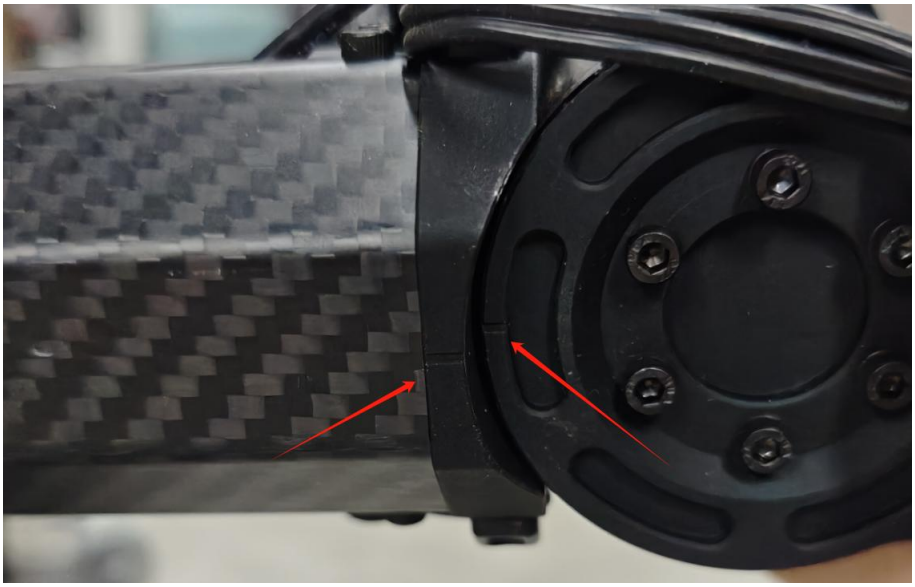
机械臂出厂时关节已经标定完成，不需要再次进行标定，若在使用过程中不慎标定的机械臂的关节，可能导致机械臂电机保护（指示灯闪烁红灯），可通过关节标定功能恢复关节参数。其中关节 1-关节 5 需要标定关节中值，关节 6 需要标定关节零点。

以关节 3、关节 6 为例，说明机械臂关节标定方法。

（1） 将机械臂 USB 数据线连接到电脑

（2） 软件自动获取机械臂串口信息并连接

（3） 手动将关节调整到中间位置，两刻度槽对齐（关节 1、关节 5 可肉眼判断中间位置）



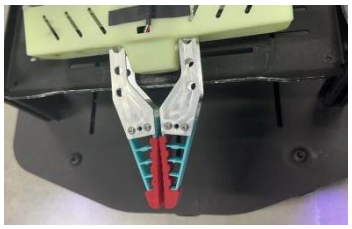
（4） 点击对应关节中值标定按钮

（5） 点击 Yes

（6） 关节 6 标定，点击标定按钮将当前位置标记为零点

（7） 点击 Yes

（8） 慢慢控制机械爪直至机械爪闭合



（9） 再次点击标定按钮标定零点

（10）点击 Yes

（11）标定完成后需要重新示教动作库

六、 软件控制

机械臂控制类位于以下路径，封装了动作库执行、单关节电机使能控制、单关机电机动作控制等函数，参赛选手可根据实际需求调用。

sebot-t710-competition\sebot_catering\src\sebot_catering\include\arm.hpp

6.1. 动作库执行

- 函数定义

```
/**
 * @brief 设置机械臂动作组
 *
 * @param action 动作类型
 */
void setActions(Action action)
{
    if (!isOpen)
        return;

    actionEnable = false;

    uint8_t buff[6];                    // 多发送一个字节
    uint8_t check = 0;                  // 校验位
    buff[0] = USB_FRAME_HEAD;           // 通信帧头
    buff[1] = USB_ADDR_JOINT_ACTION;    // 地址
    buff[2] = 5;                        // 帧长

    buff[3] = (uint8_t)(action + 1); // 动作类型

    for (int i = 0; i < 4; i++)
        check += buff[i];
    buff[4] = check; // 校验位

    // 循环发送数据
    for (size_t i = 0; i < 6; i++)
        transmitByte(buff[i]);

    actionOver[action] = false; // 状态刷新
}
```

- 参数示意

```
/**
 * @brief 机械臂动作库类型
 *
 */
typedef enum
{
    ACTION_RES = 0, // 机械臂复位
    ACTION_EXT,     // 机械臂伸展
    ACTION_CUR,     // 机械臂收缩
    ACTION_PUT,     // 机械臂放置
    ACTION_DIY      // 自定义动作
} Action;
```

- 调用案例

```
// 机械臂初始化
Talon talon;
sleep(2);

if (!talon.isOpen)
{
    ROS_ERROR_STREAM("Talon-Uart open failed!!!");
    return 0;
}
talon.setActions(talon.ACTION_EXT); // 机械臂动作：伸展
```

6.2. 单关节控制

- 函数定义

```
/**
 * @brief 机械臂单关节控制
 *
 * @param joint 关节电机编号
 * @param position 关节电机位置：-2π~2π（rad）
 */
void setJointPosition(ArmJoint joint, float position)
{
    if (!isOpen)
        return;

    if (position < 0)
        position = 0;
    else if (position > 10)
        position = 10;

    uint8_t buff[10];              // 多发送一个字节
    uint8_t check = 0;             // 校验位
    buff[0] = USB_FRAME_HEAD;      // 通信帧头
    buff[1] = USB_ADDR_JOINT_CTRL; // 地址
    buff[2] = 9;                   // 帧长

    buff[3] = (uint8_t)(joint + 1); // 关节电机编号

    Bit32Union bit32Union;
    bit32Union.float32 = position; // 声明联合体对齐数据位
    for (int i = 0; i < 4; i++)
        buff[i + 4] = bit32Union.buff[i]; // 写数据

    for (int i = 0; i < 8; i++)
        check += buff[i];
    buff[8] = check; // 校验位

    // 循环发送数据
    for (size_t i = 0; i < 10; i++)
        transmitByte(buff[i]);
}
```

- 参数说明

```
/**
 * @brief    机械臂关节编号
 **/
typedef enum
{
    ARM_JOINT_1 = 0,
    ARM_JOINT_2,
    ARM_JOINT_3,
    ARM_JOINT_4,
    ARM_JOINT_5,
    ARM_JOINT_6,
    ARM_JOINT_ALL, // 所有关节
} ArmJoint;
```

- 调用案例

```
talon.setJointPosition(talon.ARM_JOINT_6, 9.0); // 机械爪张开：9cm
```

6.3. 机械爪运动控制

- 函数定义

```
/**
 * @brief 机械爪运动控制
 *
 * @param x X轴方向角度控制
 * @param y Y轴方向角度控制
 */
void setClawMotion(float x, float y)
{
    if (!isOpen)
        return;

    uint8_t buff[13];              // 多发送一个字节
    uint8_t check = 0;             // 校验位
    buff[0] = USB_FRAME_HEAD;      // 通信帧头
    buff[1] = USB_ADDR_CLAW_MOTION; // 地址
    buff[2] = 12;                  // 帧长

    Bit32Union bit32U;
    bit32U.float32 = x;
    for (int i = 0; i < 4; i++)
        buff[3 + i] = bit32U.buff[i];
    bit32U.float32 = y;
    for (int i = 0; i < 4; i++)
        buff[7 + i] = bit32U.buff[i];

    for (int i = 0; i < 11; i++)
        check += buff[i];
    buff[11] = check; // 校验位

    // 循环发送数据
    for (size_t i = 0; i < 13; i++)
        transmitByte(buff[i]);
}
```

- 调用案例

```
talon.setClawMotion(-pidClawX.out, 0.3); // 机械爪运动控制: 往上抬0.3rad
```